HDOC Day-54

****

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k < size. Pass the array to a function named max_subarray_sum() using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function max_subarray_sum() must:

- Identify all contiguous subarrays of size k in arr[].
- Calculate the sum of the elements in each subarray.
- Find and display the maximum subarray sum.
- Display -1 if k is invalid.

1. Problem Understanding

The program reads the size, array elements, and k. It passes the array address to max_subarray_sum(). The function checks whether k is valid, examines every contiguous subarray of size k, calculates its sum, and displays the maximum sum. If k is invalid, it displays -1.

2. Standard Test Cases

Test Case	Size (n)	Array Input	k	Expected Output
1	6	2 1 5 1 3 2	3	Maximum subarray sum: 9
2	5	2 3 4 1 5	2	Maximum subarray sum: 7
3	3	1 2 3	4	-1
4	5	-2 -1 -3 -4 -1	2	Maximum subarray sum: -3
5	4	5 2 1 3	1	Maximum subarray sum: 5

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length integer array arr[n].
4. Read the array elements.
5. Read k.
6. Pass arr, n and k to max_subarray_sum() using the array address.
7. Stop.

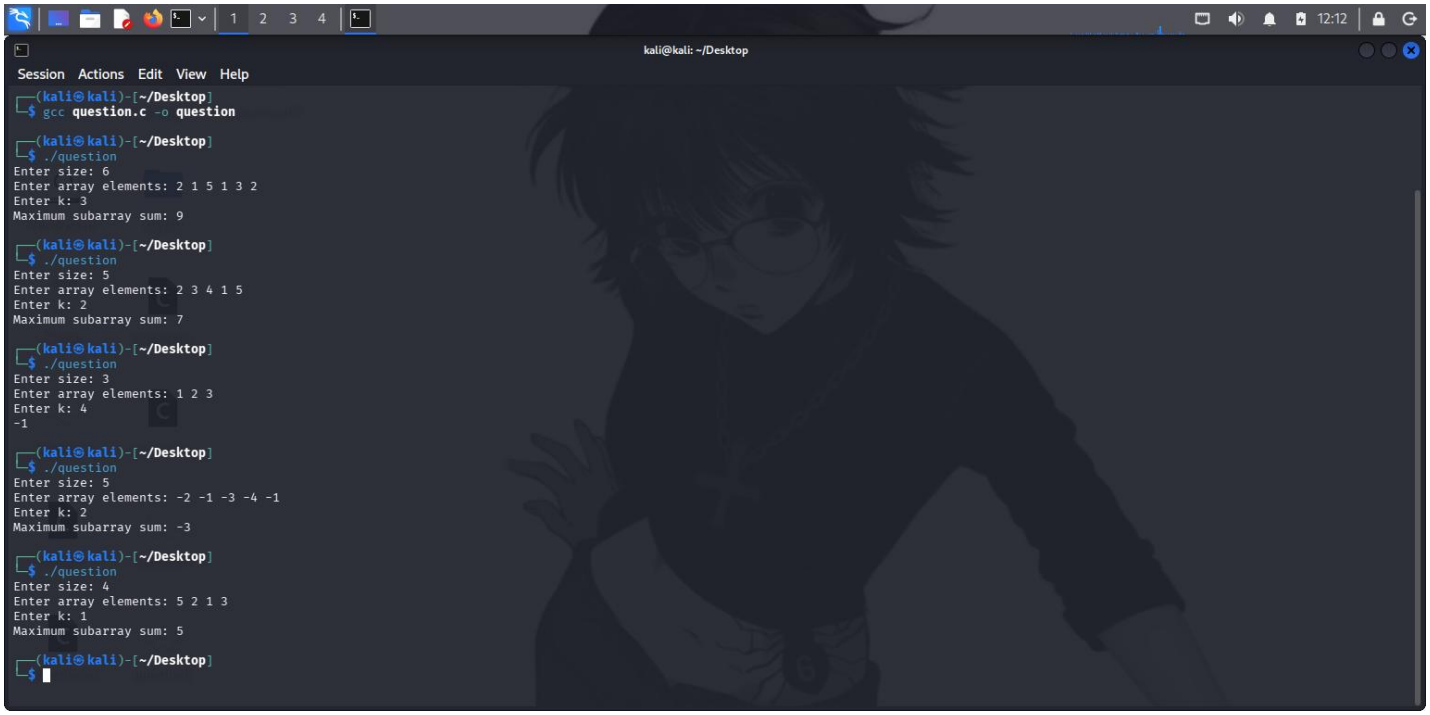
Function: max_subarray_sum()

1. If k <= 0 or k > n, display -1 and return.
2. For each possible starting index i from 0 to n-k, initialize sum = 0.
3. Add the k consecutive elements arr[i] through arr[i+k-1] using pointer notation.
4. For the first subarray, store its sum as maxSum.
5. For every later subarray, update maxSum if its sum is greater.
6. Display maxSum.

4. C Program

```
#include <stdio.h>
void max_subarray_sum(int *arr, int n, int k)
{
   int i, j, sum, maxSum = 0;
   if(k <= 0 || k > n)
   {
      printf("-1\n");
      return;
   }
   for(i = 0; i <= n - k; i++)
   {
      sum = 0;
      for(j = 0; j < k; j++)
      {
         sum = sum + *(arr + i + j);
      }
      if(i == 0 || sum > maxSum)
      {
         maxSum = sum;
      }
   }
   printf("Maximum subarray sum: %d\n", maxSum);
}
int main()
{
   int n, i, k;
   printf("Enter size: ");
   scanf("%d", &n);
   int arr[n];
   printf("Enter array elements: ");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }
   printf("Enter k: ");
   scanf("%d", &k);
   max_subarray_sum(arr, n, k);
   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	k	Actual Result	Status
1	6	2 1 5 1 3 2	3	Maximum subarray sum: 9	Pass
2	5	2 3 4 1 5	2	Maximum subarray sum: 7	Pass
3	3	1 2 3	4	-1	Pass
4	5	-2 -1 -3 -4 -1	2	Maximum subarray sum: -3	Pass
5	4	5 2 1 3	1	Maximum subarray sum: 5	Pass